

Rapport TTK4235

Heislab av Gruppe 12

1. Introduksjon

I denne rapporten reflekterer vi over utviklingsprosessen ved vår løsning av heislabben i emnet TTK4235. Utviklingsprosessen baserer seg på V-modellen. For å gjennomføre V-modellen har vi brukt UML, og designet arkitekturdiagram, sekvensdiagram, klassediagram og tilstandsdiagram. Andre verktøy vi har brukt er GitHub, Doxygen, Drawio og PlantUML. Vi dreier ut om vår opplevelse av de forskjellige stegene i V-modellen. Til slutt forklarer vi vår minimale bruk av KI, og reflekterer over gode og dårlige måter KI kan brukes til å løse dette prosjektet.

2. V-modellen

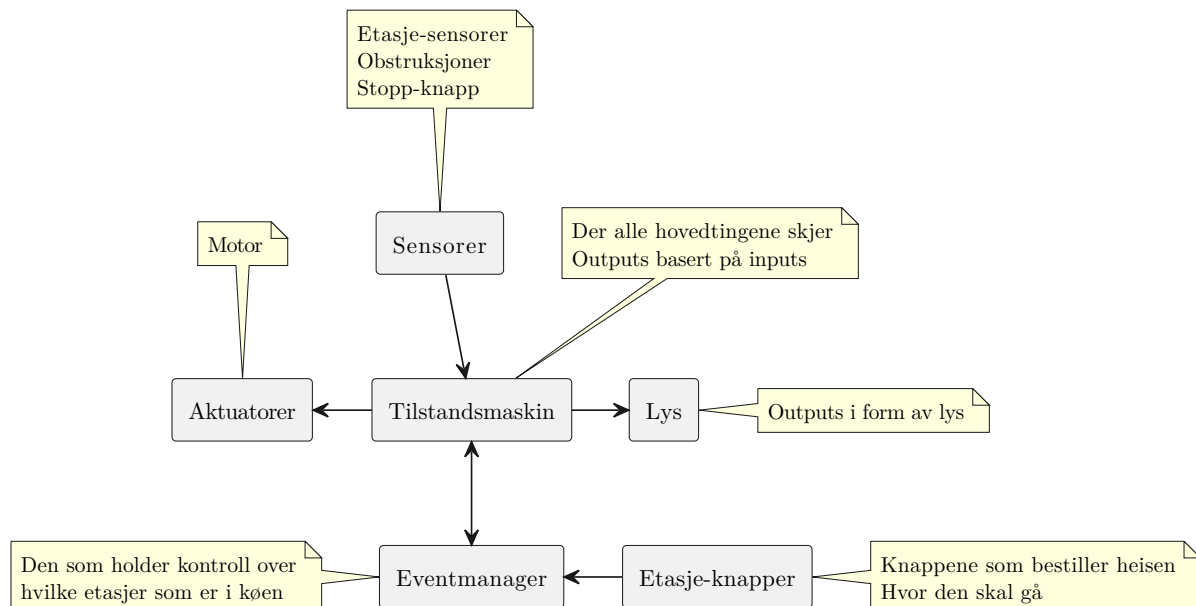
V-modellen var en sentral del av planleggingen og gjennomførelsen av arbeidet. En av oss har tidligere brukt V-modellen for å implementere et system så derfor er ikke tanken ny for oss. Å faktisk sette seg inn i de ulike punktene som den består av gjorde at vi lettere kunne implementere modulene når man faktisk skal gjøre arbeid. Vi har erfart at det å tenke overordnet over hvordan systemet skal fungere på et høyere nivå og hvilke moduler vi trenger har hjulpet oss senere når det var tid for programmering.

Vi startet først å lage alle UML-diagrammene våre i drawio, men når vi kom til tilstandsdiagrammet innså vi hvor komplisert det ville være å manuelt plassere hver eneste blokk og linje, derfor fikk vi høre fra en annen gruppe om et program som heter PlantUML som vi da brukte til å lage diagrammene. Dette gikk såpass mye fortere å bruke at vi derfor valgte å lage flere av tegningene på nytt utenom sekvensdiagrammet. Derfor er det litt ulikt format på de ulike diagrammene.

2.1. Arkitekturdesign

Den første delen av V-modellen er arkitekturdesign, her er tanken at man ser på systemet helt overordnet og ikke tenker på detaljer. Derfor lagde vi tre ulike diagrammer som har ulike abstraktsnivåer. I arkitekturdesign delen av prosessen skal man også planlegge for overordnet helhetstesting av systemet, men ettersom vi har fått utlevert en FAT-test så er ikke dette noe som vi selv må planlegge for.

2.1.1. Arkitekturdiagram

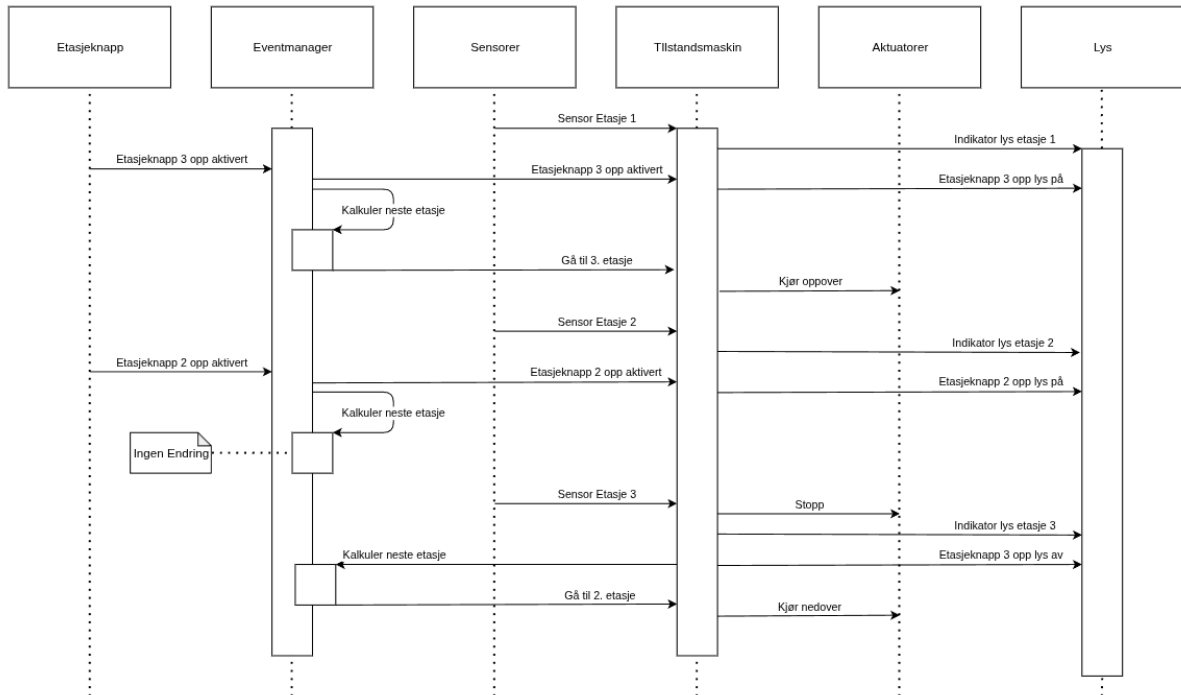


Arkitekturdiagram

Arkitekturdiagrammet er så overfladisk man kommer, her setter man opp modulene man skal ha og hvordan de skal snakke sammen uten å tenke på hvordan de faktisk skal utføres. Vi syntes dette var et veldig lett diagram å lage og var veldig praktisk under konstruksjon av de neste diagrammene. Det var derimot sjeldent at vi åpnet dette diagrammet etter konstruksjon av klassediagrammet ettersom det

hadde samme struktur, men hadde flere detaljer enn arkitekturdiagrammet. Vi valgte å splitte opp I/O i flere moduler grunnet at det virket lettere å differensiere mellom dem ettersom de har ganske ulike oppgaver. Og dersom vi ikke gjorde det hadde I/O blitt en gigantisk blokk som hadde blitt uoversiktlig. Vi valgte også å ha en eventmanager, jobben til denne var bare å se hvilken vei vi gikk og holde en oversikt over hvilke knapper som ble trykket inn. Dette er slik at den kunne sende korrekt “Neste etasje” og “Nåværende etasje” til tilstandsmaskinen uten at tilstandsmaskinen også trenger å ha denne implementasjonen.

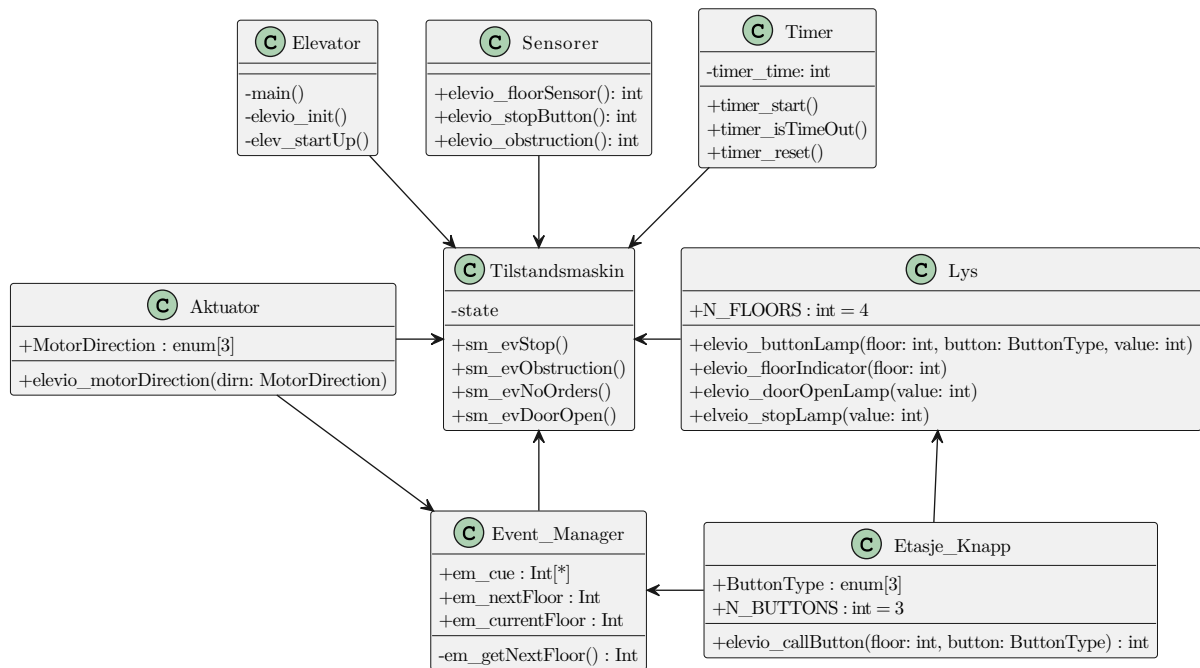
2.1.2. Sekvensdiagram



Sekvensdiagram

Sekvensdiagrammet er et diagram som viser et visst scenario i en lineær tidsperiode. Diagrammet kunne vært mye mindre komplisert, men vi valgte å ta for oss en litt mer avansert situasjon ettersom vi har begge kjørt masse heis før og følte ikke at det var nødvendig å bruke tid på å implementere ett lett sekvensdiagram som tok for seg helt normal og forventet oppførsel. Dette diagrammet hjalp oss godt med å få en tanke om hvordan oppførselen til heisen faktisk skulle være og hvordan den skulle håndtere flere bestillinger på en gang. Dette diagrammet ble ikke brukt mye etter at det var laget, men hjalp i starten for å samle tanker om hvordan heisen skulle oppføre seg.

2.1.3. Klassediagram



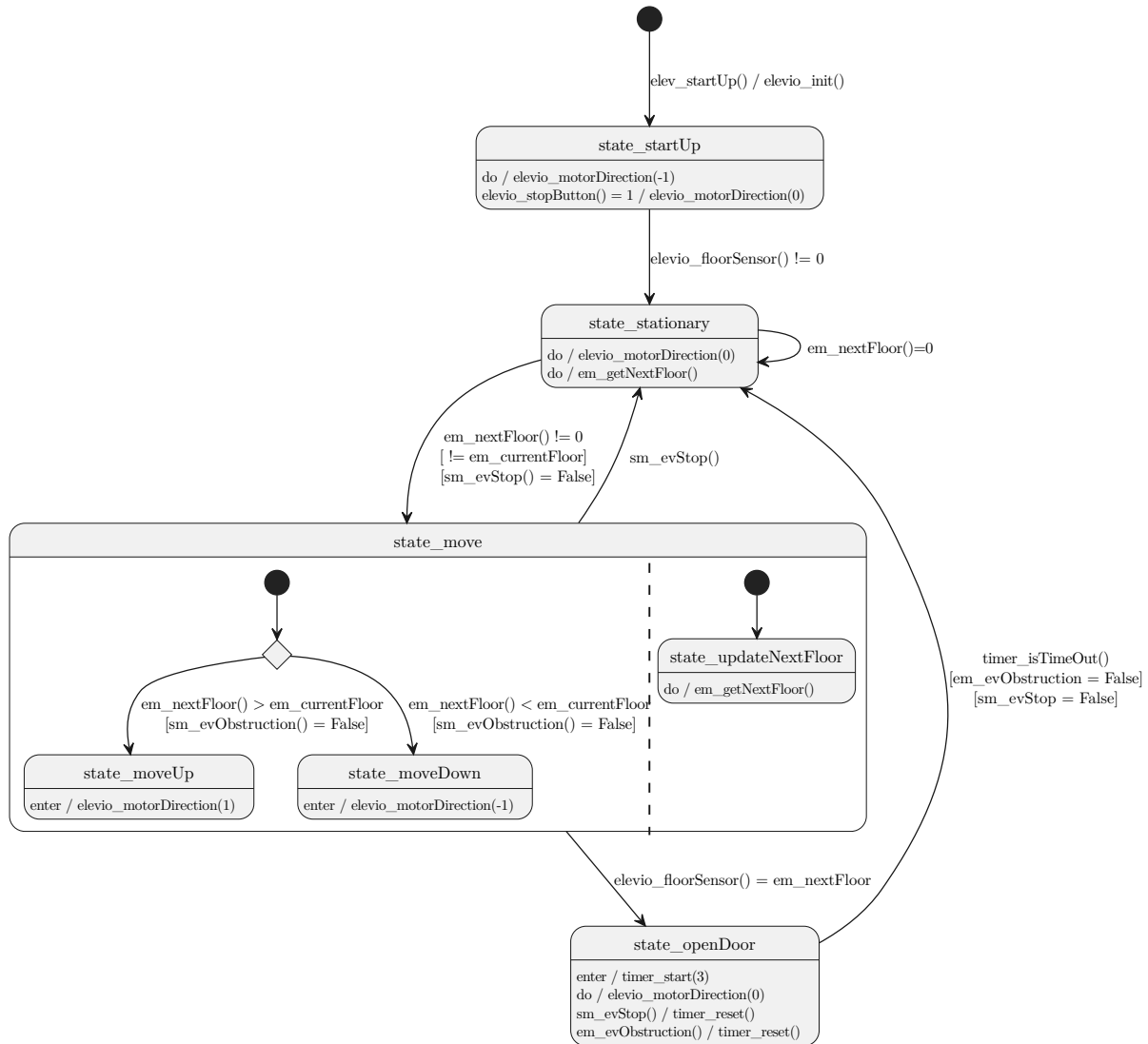
Klassediagram

Klassediagrammet er et mer komplisert diagram og kan ha ulike abstraksjonsnivåer ut ifra hvor mye tid man ønsker å bruke på det. Vårt klassediagram ser ut til å ha mange funksjoner, men de fleste er allerede implementerte funksjoner i utdelt kode så vi tok derfor å bare kopierte det som sto der og brukte det. Dette er det desidert mest brukte og praktiske diagrammet i vårt reportoar, grunnen til dette er at her står alle funksjonene vi tror vi trengte på dette tidspunktet og den sier noe om hvem som arver fra hvem. Vi endte opp med å gjøre endringer på denne etterhvert i prosessen når vi innså at vi hadde tenkt feil på flere av funksjonene, men den var fremdeles ekstremt brukbar ettersom den overordnede strukturen ble beholdt. Det var også ekstremt lett å gjøre endringer ettersom vi brukte et program som gjorde at man ikke trengte å endre størrelser og plasseringer manuelt.

2.2. Moduldesign

I fasen moduldesign er tanken å konkretisere en del av hvordan modulene faktisk oppfører seg uten å ta med hvordan de skal programmeres. Klassediagrammet over hadde en såpass grei mengde med detaljer at vi gikk rett på tilstandsdiagrammet for å finne ut av hvordan man skulle programmere tilstandsmaskinen vår. Man kan argumentere for at klassediagrammet også har en del moduldesign i seg, men ettersom V-modellen ikke nødvendigvis har harde overganger fløt disse litt inn i hverandre. Vi valgte kun å lage tilstandsdiagram ettersom vi tenkte at implementasjon av **Event_Manager** skulle være lett å gjøre. Her tok vi litt feil ettersom at vi havnet på implementasjon uten å konkret ha tenkt over hvordan den egentlig skulle oppføre seg.

2.2.1. Tilstandsdiagram



Tilstandsdiagram

Tilstandsmaskinen har her litt andre funksjoner enn tidligere spesifisert og grunnen til dette var at vi tidlig så at de tidligere definerte funksjonene ikke ga mening. Det å splitte opp i en **state_startUp** fase som den kun er i i starten gjorde at oppstarten av heisen ble lettere å implementere, hvor det rett og slett bare var å ignorere alle knapper og bare finne ut av hvor heisen var hen.

Vi valgte også å ha to faser hvor **state_openDoor** har åpen dør og **state_stationary** har lukket dør. Etterhvert innså vi at vi måtte ha en kobling fra lukket dør til åpen dør direkte dersom den sto stille i en etasje og noen ønsket å komme inn i heisen. Slik som vi har satt det opp så er tilstandsmaskinen helt avhengig av at **Event_Manager** sier noe om hva som skal gjøres.

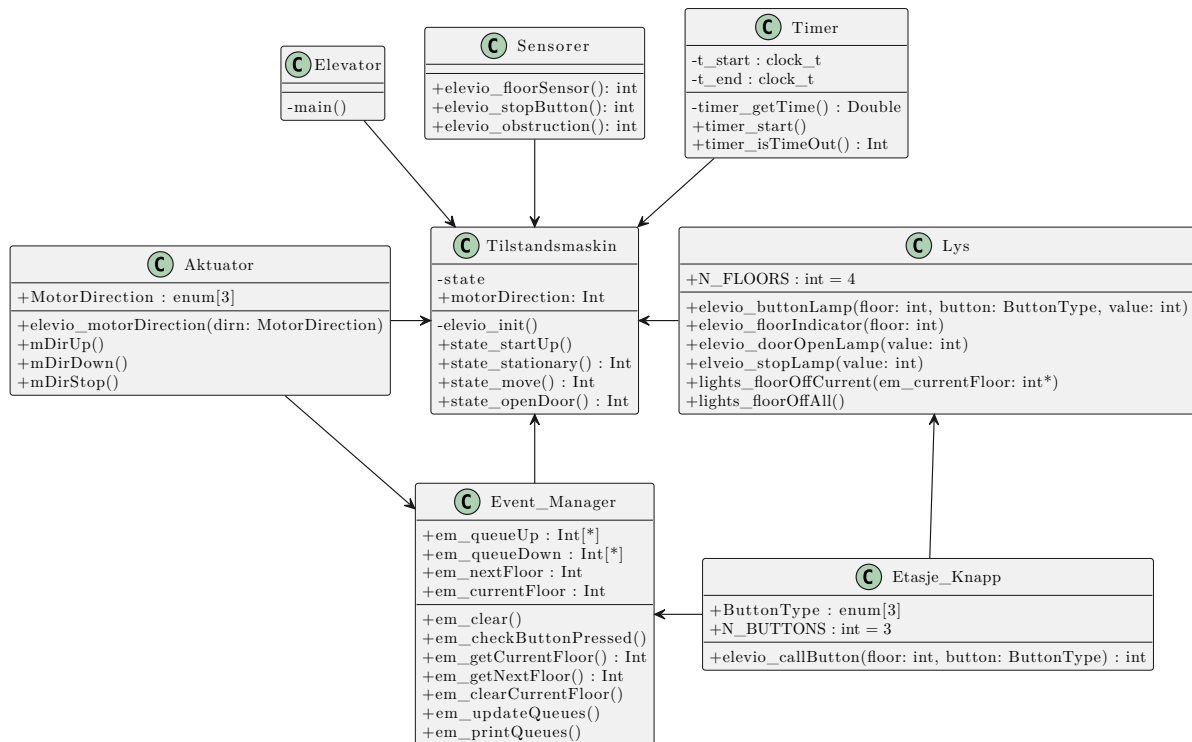
Inne i **state_move** har vi noen ulike tilstander, men under implementasjon så valgte vi å ikke beholde at dette er ulike tilstander ettersom det krever parallellisering av å bevege seg og å sjekke om vi skal stoppe underveis.

2.3. Implementasjon

Implementasjonsfasen begynte relativt sent for vår del. Dette følger fra oppsettet til V-modellen, ettersom målet var at vi skulle kartlegge alt av logikk og funksjonalitet i forveien av selve implementasjonen. Det å bestemme funksjonaliteten før implementasjonen gjorde det lettere å programmere i et språk vi hittil ikke hadde brukt noe særlig før.

Noen endringer i funksjonaliteten forekom under implementasjonen av programmet vårt. Dette oppsto som oftest i situasjoner hvor vi innså bedre måter å løse et problem, enn det vi gjorde før vi testet det i praksis. Dette kan skyldes V-modellens rigide struktur, samt det kan også forklares av vår egen mangel på erfaring. Ved å utvikle på denne måten flere ganger, vil en få flere erfaringer og kunnskap rundt hvordan systemer burde designes og kan i teorien ende opp med færre endringer etter implementasjonsfasen har begynt.

2.3.1. Endring i Klassediagram



Nytt Klassediagram

Klassediagrammet fikk spesielt store endringer. Ved å sammenlikne med det originale klassediagrammet ser en at det har vokst betraktelig under implementasjonsfasen. Dette skjedde ofte da vi innså hvordan `elevio`-funksjonene fungerte, og kom på praktiske funksjoner for bruk av disse heis-funksjonene.

`Event_Manager` vokste spesielt mye. Her er det tydelig at vi endret løsningen for bestillingskøen vår, ettersom det nå er en kø som husker bestillingene oppover og en som husker bestillingene nedover. I tillegg kom vi på andre gode måter å behandle bestillingskøen på, og lagde derfor flere funksjoner som gjorde det enklere å designe heisen etter kravene.

I tillegg er det også tydelig at noen funksjoner ikke har havnet i den klassen de burde være, og da spesielt i `Event_Manager`. `em_checkButtonPressed()` er en funksjon som hadde passet bedre i `Etasje_Knapp`, og `em_getCurrentFloor()` er en funksjon som burde vært under `Sensorer`. Disse havnet nok her fordi fokuset under implementasjonsfasen nettopp var implementasjon, og ikke klassesdesign. Ettersom `em_queueUp` og `em_queueDown` nå ligger i `Event_Manager` istedenfor i `Tilstandsmaskin`, vil nå denne klassen også ha minne. Dette var ikke slikt vi originalt tenkte det, men denne versjonen fungerer også. Det er tross alt bare å flytte to arrays fra en klasse til en annen, uten noen større endringer i koden enn navneskift.

2.3.2. GitHub

Vi valgte å bruke GitHub for å sørge for at implementasjonen og samarbeid mellom oss gikk på skinner. Git er et ekstremt brukbart verktøy ettersom man kan gå tilbake i tid dersom noe har fungert før, men ikke fungerer lenger. I tillegg så er det å kunne gjøre endringer parallelt og kjøre det uten at koden er komplett på hver vår maskin gjorde at vi kunne jobbe på to helt ulike systemer parallelt. Ettersom vi også ikke skrev i samme filer samtidig hadde vi så og si ingen merge-konflikter, og når vi først hadde det, valgte vi som regel bare å skrive over hverandre ettersom det som regel ikke var meningen å pushe alle endringene.

2.3.3. Doxygen

Vi brukte Doxygen for å dokumentere funksjonaliteten til de forskjellige funksjonene i koden vår. Dette fungerte bra, siden vi da fikk ett samlet sted med informasjon om hvordan bruke tidligere implementerte funksjoner. Dette virker som et bra verktøy for større team, hvor noen skriver funksjoner som andre trenger og tiden mellom du skrev en funksjon og du trenger den kan være lang. For vår del var vi kun to stykker i et prosjekt på relativt kort tid, så det letteste var kun å snakke med hverandre om kode den andre hadde skrevet. Vi leste derfor ikke vår egen Doxygen så mye under utviklingen, men ser at det kan være spesielt nyttig dersom vi skal bruke koden vår igjen en gang senere.

2.4. Enhetstesting

Under enhetstesting så tester man hver enkelt modul og sørger for at oppførselen er som forventet. Dette gjorde vi lett ved å først kalle hver enkelt funksjon fra `main()` og sørge for at returverdier og endringer dersom det var “pass by reference” ble gjort korrekt.

De mer kompliserte modulene krevde en mer generell og overordnet testing og vi var derfor nødt til å bruke den utleverte simulatoren for å teste at det man gjorde ble korrekt.

Ettersom vi var litt sent ute på å programmere endte vi opp med å måtte implementere og teste alt via simulatoren uten å vite nøyaktig om dette ville fungere i virkeligheten. Dette gjorde at usikkerheten rundt om det faktisk fungerte var høy.

Vi møtte på færre problemer enn forventet når vi testet og det gikk derfor på skinner.

Vi endte opp med å hoppe opp og ned fra modultesting og implementasjon ettersom det å skrive ferdig all koden til hele heisen uten testing ville bare gjort oppgaven unødvendig komplisert. Så når en del var ferdig brukte vi modultesting for å sjekke at modulen fungert før vi gikk videre til neste.

2.5. Integrasjonstesting

For full integrasjonstesting så hadde vi fått utlevert en FAT så derfor var testingen rett og slett bare å gå gjennom denne FAT-en og sørge for at oppførselen var som forventet.

Noe som vi opplevde under testen var at vi ittererte gjennom knappene fra 1 til 4 og returnerte når den fant en knapp. Derfor om man holdt inne knappen helt nede ville ikke heisen lenger registrere nye bestillinger. Dette ville vært lett å fikse ettersom vi hadde “pass by reference” og det derfor bare var å fortsette å iterere igjennom listen og ikke stoppe når den finner en knapp som aktiverer seg.

3. Bruk av KI

Under labarbeidet brukte vi KI til å hjelpe oss med å installere PlantUML, da vi hadde problemer med dette og ikke mente det ville gå utover ønsket læringsutbytte.

Vi innser at det hadde vært mulig å løse hele oppgaven ved hjelp av KI. Selv om heisen kunne fungert akkurat slik den skulle, vil dette ikke være ønskelig. Læringsutbyttet fra å la KI løse oppgaven er betydelig lavere enn å løse oppgaven selv, og dette er en studieoppgave hvor læringsutbyttet er særdeles viktig.

Bruk av KI kan, ved bruk av kybernetiske begrep, klassifiseres som åpen- eller lukket-sløyfe. Åpen-sløyfe vil bety at du stoler blindt på det KI-en forteller deg, og ikke sjekker om det faktisk stemmer. Ved å sjekke om det KI-en forteller deg stemmer, vil du få en tilbakekobling på informasjonen du tar til deg, og dermed oppnå lukket-sløyfe. Siden vi brukte KI for hjelp med installasjon av en utvidelse, og ikke til forklaring av pensum eller hjelp med programmering, vil dette klassifiseres som lukket-sløyfe. Dette er fordi det er umiddelbart åpenbart om hvorvidt PlantUML faktisk har blitt installert, ved at den ellers ikke ville fungert.

4. Refleksjon

Generelt var dette en lærerik og god oppgave for å friske opp i både programmering og for å se hvordan man i virkeligheten ville implementert et større system. Det å bli tvunget til å planlegge godt var en god øvelse, ettersom tidligere så har man som regel bare kastet seg på oppgaver og om det ikke fungerer blir det mye større jobb å rette opp i feil. Oppgaven var også god til å faktisk lære oss om ulike bruk av UML-diagrammer og ikke bare få en beskrivelse av dem i forelesning.

Selve programmeringsdelen for vår del gikk ekstremt fort, den største kritikken vi har til vår egen kode er at vi ikke implementerte en `typedef struct elevator` slik at vi kunne sende verdier mellom funksjoner uten å måtte skrive ekstremt stygg kode. Dette burde være relativt rask å endre på, og er selvfølgelig noe vi kommer til å ta med oss senere. Men til tross for noen dårlige programmeringsnotasjoner så gikk oppgaven bra og vi føler på god mestring.